Remove Adjacent Duplicate Characters

Problem Description

You are given a String s containing lowercase English letters. Your task is to repeatedly **remove two identical characters** that appear next to each other. Whenever two adjacent characters are the same, both characters must be removed. Continue until no adjacent duplicate characters remain.

Return the final String. You must use a Stack to solve the problem without using arrays.

Examples

Example 1

Input	`s = "abbaca"`
Output	`"ca"`

Explanation: Remove "bb" to obtain "aaca". Then remove "aa" to obtain "ca".

Example 2

Input	`s = "azxxzy"`
Output	`"ay"`

Explanation: Remove "xx" to obtain "azzy". Then remove "zz" to obtain "ay".

Example 3

Input	`s = "abcd"`
Output	`"abcd"`

Explanation: No identical characters appear next to one another, so the String is unchanged.

Example 4

Input	`s = "aabbcc"`
Output	`""`

Explanation: All adjacent duplicate pairs disappear, leaving an empty String.

Bonus Challenge (+ Marks)

Modify the program to repeatedly remove groups of **three identical adjacent characters** instead of two. Continue until no group of three identical adjacent characters remains.

Input: `s = "deeedbbcccbdaa"`

Output: `"aa"`

Constraints

- 1 ≤ s.length() ≤ 100,000.
- s contains only lowercase English letters (a–z).
- Use Stack<Character>; arrays are not allowed.
- The solution must run in O(n) time.

Java Starter Code

```
import java.util.Stack;

class Solution {
    public String removeDuplicates(String s) {
        Stack<Character> stack = new Stack<>();

        // TODO 1: Read the characters one by one.
        // TODO 2: Compare with the top of the stack.
        // TODO 3: Pop if they match; otherwise push.
        // TODO 4: Build and return the final String.

        return "";
    }
}
```

Student Requirements

- Use Stack<Character> to store characters.
- Use push(), pop(), peek(), and isEmpty() where appropriate.
- Remove adjacent duplicate pairs as they appear; this may create new duplicate pairs.
- Return the remaining characters in the original order.
- Do not use arrays, recursion, replace(), or replaceAll(). StringBuilder is allowed.

Submission: Submit the completed Java Solution class and briefly explain how the Stack's last-in, first-out (LIFO) principle helps remove adjacent duplicates.

Students must complete the assignment, upload their Java source code to GitHub, and share the repository link with the class monitor **before Thursday at 11:00 AM**. **Each student must fully understand their code and be able to explain its logic, stack operations, and how it works**. Late submissions will not be accepted.